

Data Types

The Building Blocks of Memory



Core Elements, Memory Sizes, and User Definitions

BASIC • DERIVED • USER-DEFINED

1. Primitive Data Types

Primitive types are the standard types built into the C++ compiler. They represent single values and are the foundation for all complex data structures.

Type	Typical Size	Description
int	4 bytes	Integer / Whole numbers
float	4 bytes	Single-precision decimal
double	8 bytes	Double-precision decimal
char	1 byte	Single character/ASCII
bool	1 byte	Logical true/false

Note: Sizes may vary slightly between 32-bit and 64-bit systems, but these are the standard C++ minimums.

2. Type Modifiers

You can change the size and range of basic types using modifiers to match your specific memory requirements.

- **short:** Restricts size (usually 2 bytes).
- **long:** Increases size (usually 8 bytes).
- **signed:** Allows positive and negative values.
- **unsigned:** Only allows positive values (doubles max range).

```
unsigned int positiveOnly = 5000;  
long double extremePrecision = 3.1415926535;
```

3. Text Presentation

The char type

Stores a single character using single quotes. Internally, it is stored as an integer (ASCII value).

```
char letter = 'A'; // ASCII 65
```

The string class

While often thought of as a data type, `string` is actually a class in the Standard Library used for text.

```
#include <string>
string welcome = "Welcome to CodeHive";
```

4. Boolean Logic

The `bool` type is essential for decision making. It can only hold **true** (1) or **false** (0).

```
bool isCourseFull = false;
if (isCourseFull) {
    cout << "Registration closed.";
}
```

Memory Tip: Even though a boolean only needs 1 bit to represent 0 or 1, most compilers allocate 1 full byte to ensure each variable has a unique address.

5. Decimals: Float vs Double

Choosing between float and double depends on how much accuracy your math requires.

- **float:** Accurate to ~7 decimal places. Always add an 'f' suffix (e.g., 3.14f).
- **double:** Accurate to ~15 decimal places. This is the default in C++.

```
float gpa = 3.85f;  
double pi = 3.141592653589793;
```

6. Derived Data Types

These types are built by combining or pointing to primitive types.

- **Arrays:** Sequence of elements of the same type. `int arr[5];`
- **Pointers:** Variables that store the address of another variable. `int* p = &var;`
- **References:** An alias/alternate name for a variable. `int& ref = var;`

```
int score = 100;  
int &ref = score; // ref is now 100
```

7. User-Defined Types

C++ allows programmers to create their own types to model real-world data structures.

Structs

Groups different variables under one name. Used for data-only structures.

```
struct Student {  
    int id;  
    string name;  
};
```

Classes

The core of OOP. Combines data (attributes) with functions (methods).

8. Specialized Types

Enums (Enumerations)

Defines a new type with a set of named constants, improving code readability.

```
enum Difficulty { EASY, MEDIUM, HARD };  
Difficulty d = EASY;
```

The Void Type

Represents the absence of type. Used in function return types when the function doesn't return a value.

9. Summary Checklist

- **Precision:** Use `double` for most math unless memory is extremely tight.
- **Naming:** Variable names should reflect the type (e.g., `isReady` for `bool`).
- **Suffixes:** Use `f` for floats and `U` for unsigned ints.
- **Initialization:** Always initialize variables to avoid "garbage" values.

CodeHive Skill Test

Identify the best data type for:

1. Number of students in a class.
2. Price of a course.
3. Student's middle initial.
4. Precise coordinates of a GPS location.

ARCHITECTURE DEFINED: DATA TYPES MASTERED